UNIVERSIDAD PANAMERICANA

Facultad de ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y TIC

Análisis y Diseño de Sistemas

Análisis y Diseño de Sistemas

(Documento de Diseño de Arquitectura)

Kevin Eleazar Azurdia Chamalé (000137188)

Juan Diego Wosbely Sac López (000149360)

Chimaltenango, Chimaltenango, marzo de 2026

Tabla de contenido

Capítulo III: Documento de Diseño de Arquitectura - P3 ***4***

Introducción **4**

Diagrama de componentes **4**

Descripción de componentes **5**

Vista lógica y arquitectura de software **5**

Diagrama de capas **5**

Descripción de la seguridad y credenciales **7**

Diagrama de seguridad **7**

Vista de implementación **9**

Descripción General de la Implementación **9**

Estrategia de implementación: 9

Diagrama de la Implementación **11**

Vista de puesta en marcha **12**

Vista de datos **13**

Diagrama Entidad-Relación **14**

Diccionario de Datos **15**

Tabla: Roles 15

Tabla: Usuarios 15

Tabla: Publicaciones 15

Tabla: Reacciones 16

Tabla: Comentarios 17

Tabla: Hashtag 17

Tabla: publicación_hashtag 18

Definición de roles de la aplicación **20**

Limitaciones del proyecto: **20**

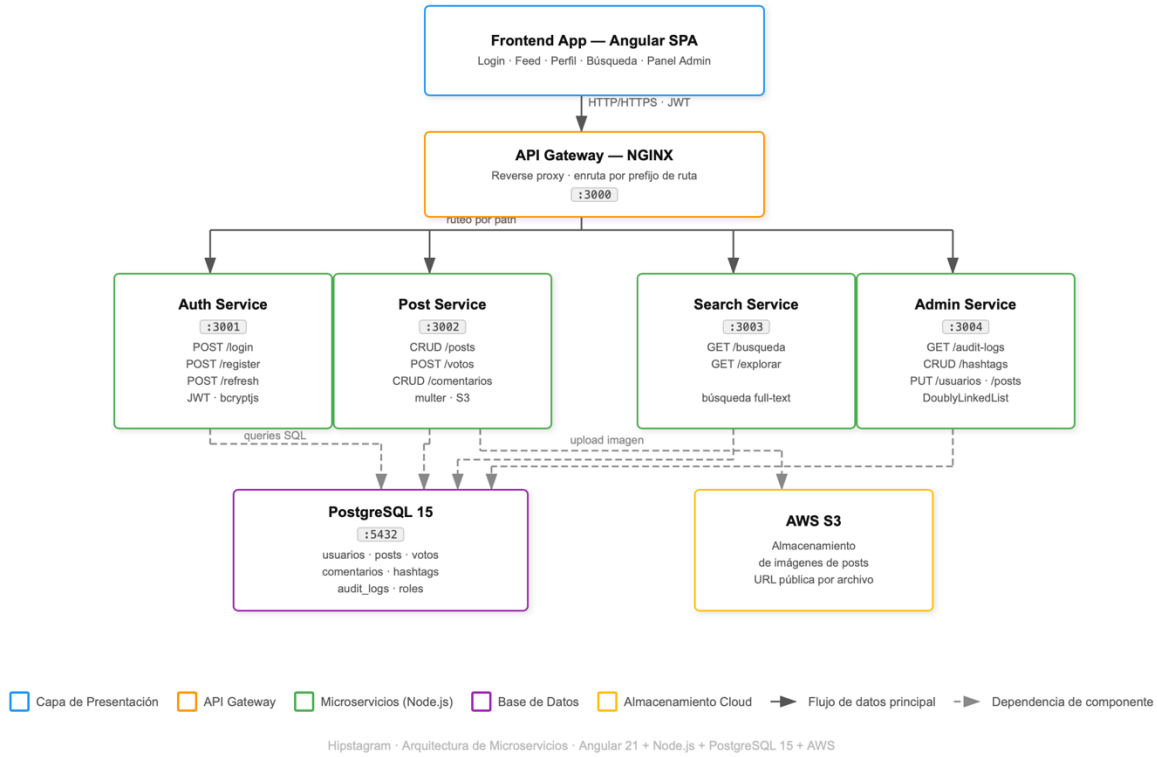
Documento de Diseño de Arquitectura - P3

Introducción

El presente Documento de Diseño de Arquitectura tiene como propósito definir y documentar la estructura técnica del sistema Hipstagram. Este documento proporciona una visión integral de la solución de software, detallando como se organizan sus componentes, la forma en que interactúan entre sí y las decisiones de diseño fundamentales que garantizan el cumplimiento de los requerimientos funcionales y atributos de calidad (escalabilidad, seguridad y disponibilidad). La arquitectura propuesta se basa en un patrón de microservicios, contenerización con Docker y un despliegue proyectado en la infraestructura de nube de Amazon Web Services (AWS).

Diagrama de componentes

A continuación, se presenta el diagrama de componentes del sistema Hipstagram, el cual ilustra la disposición estructural de los microservicios, el API Gateway, la interfaz de usuario y la capa de persistencia de datos. Este modelo visualiza como interactúan los diferentes modulos de software de forma independiente mediante sus interfaces de comunicación para cumplir con las funcionalidades de la plataforma.



Descripción de componentes

La arquitectura del sistema se divide en los siguientes componentes de software independientes:

- Fronted App (Cliente): Aplicación de interfaz de usuario (Web/Móvil) encargada de la presentación visual y la captura de interacciones de usuario.
- API Gateway: Componente centralizador que actúa como proxy inverso. Recibe todas las peticiones del cliente, valida la autenticación inicial y enruta el tráfico hacia el microservicio correspondiente.
- Auth Service (puerto 3001): Microservicio responsable de la gestión de identidades, registro de usuarios, validación de credenciales y emisión de tokens JWT.
- Post Service (puerto 3002): Gestiona publicaciones, votos (like/dislike con toggle, UNIQUE por usuario+post) y comentarios paginados. Almacena imágenes en AWS S3 vía multer-s3 y usa transacciones con SAVEPOINT para proteger la inserción de hashtags.
- Search Service (puerto 3003): Microservicio de descubrimiento que indexa descripciones y hashtags para resolver consultas de texto libre y exploración por popularidad.
- Admin Service (puerto 3004): Microservicio de administración y moderación. Gestiona usuarios, modera publicaciones, administra la lista de hashtags prohibidos y expone el registro de auditoría (DoublyLinkedList en memoria con respaldo en PostgreSQL).
- Bases de Datos (PostgreSQL & S3): Componentes de persistencia aislados; PostgreSQL para datos relacionales y AWS S3 para el almacenamiento de objetos estáticos (fotografías).

Vista lógica y arquitectura de software

Desde una perspectiva lógica, Hipstagram adopta un estilo arquitectónico de Microservicios distribuidos. En lugar de un monolito central, el sistema se descompone en pequeños dominios de negocios autónomos.

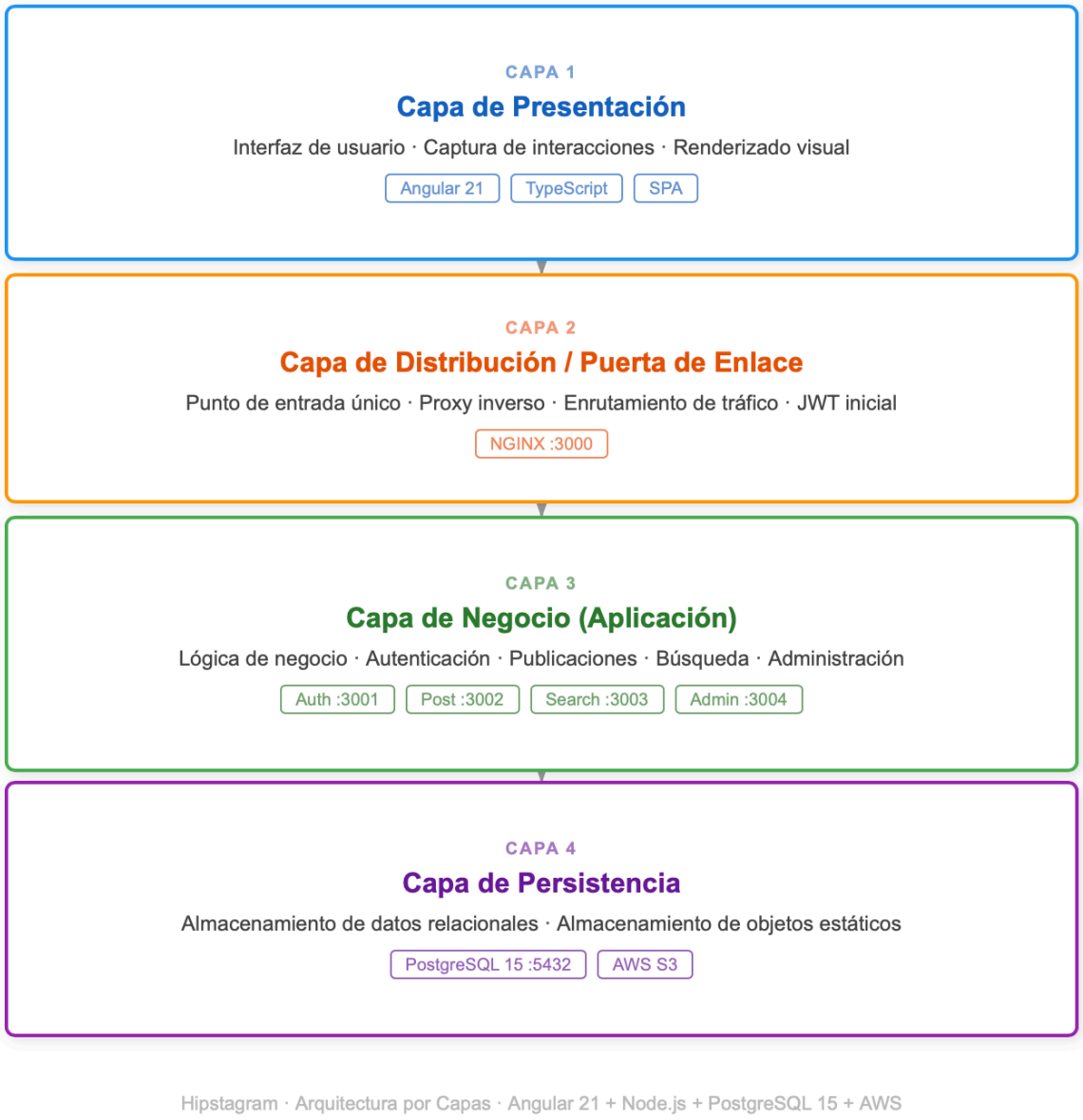
La comunicación externa (Cliente a Sistema) se realiza a través de llamadas síncronas utilizando el protocolo HTTP/REST mediado por el API Gateway. La comunicación entre microservicios se realiza de forma síncrona utilizando el protocolo HTTP/REST mediado por el API Gateway. Cada microservicio posee su propio esquema de base de datos dentro de PostgreSQL, manteniendo aislamiento lógico y evitando el acceso directo entre servicios.

Diagrama de capas

La separación de responsabilidades lógicas del sistema se estructura en un modelo de arquitectura por capas:

1. Capa de Presentación: Interfaz de usuario (Frontend)

2. Capa de Distribución / Puerta de Enlace: API Gateway.
3. Capa de Negocio (Aplicación): Ecosistema de Microservicios.
4. Capa de Persistencia: Bases de datos y almacenamiento de objetos.



Descripción de la seguridad y credenciales

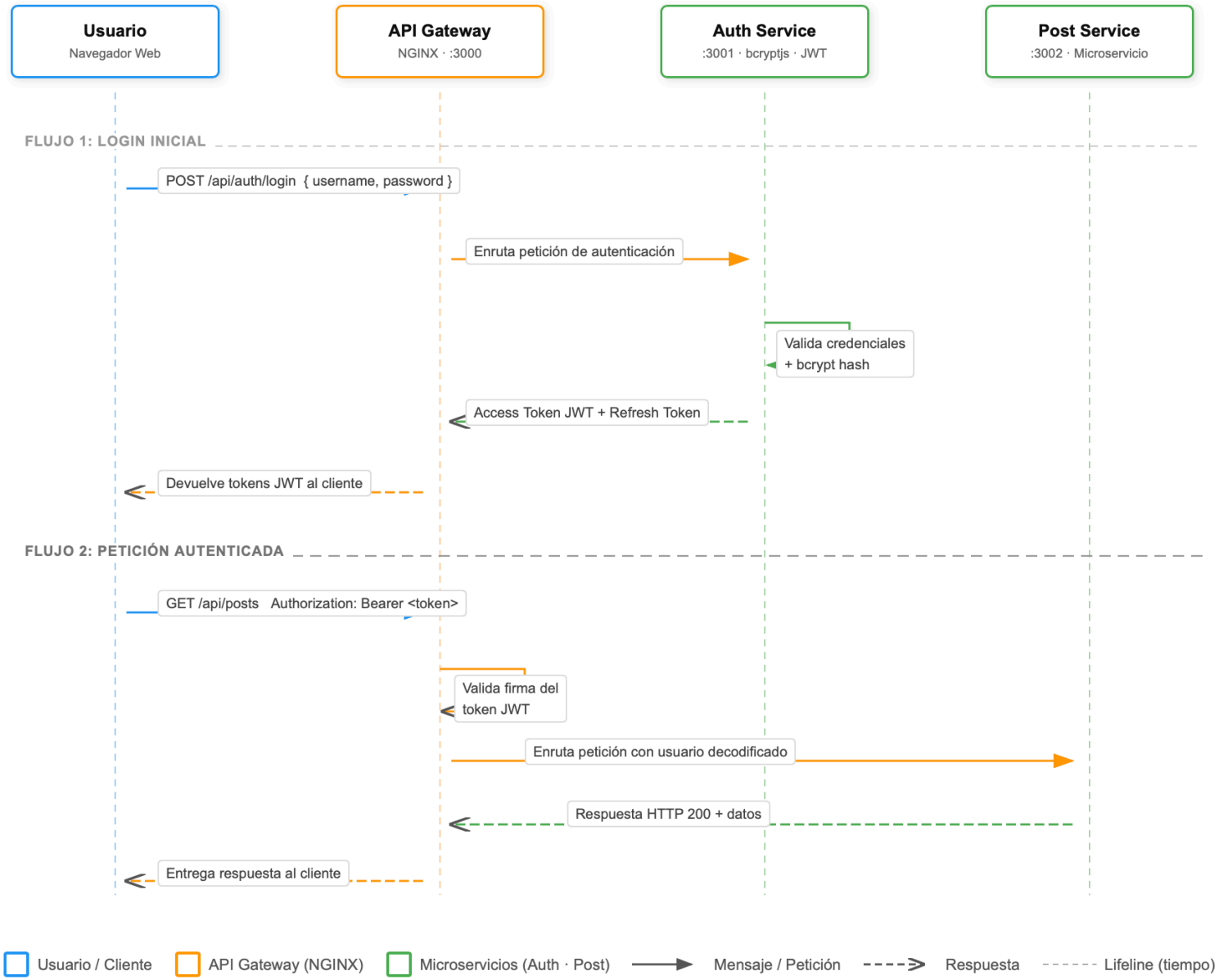
La seguridad del sistema Hipstagram se gestiona bajo un enfoque de defensa en profundidad, garantizando la confidencialidad, integridad y disponibilidad de los datos:

- Autenticación (JWT): Se utiliza el estándar JSON Web Token para el manejo de sesiones stateless. Una vez que el usuario valida sus credenciales, el Auth Service emite un token firmado que el cliente web o móvil debe incluir en el encabezado (Authorization: Bearer <token>) de cada petición posterior.
- Cifrado de Credenciales: Las contraseñas de los usuarios nunca se almacenan en texto plano. Se emplea el algoritmo de hashing estandarizado Bcrypt antes de persistirlas en la base de datos PostgreSQL.
- Autorización por Roles (RBAC): El sistema implementa un Control de Acceso Basado en Roles. El token JWT emitido contiene el rol del usuario (USER o ADMIN). El API Gateway y los microservicios validan el acceso a las funcionalidades del sistema desde la lógica del Backend.
- Protección Perimetral: El API Gateway funge como la única puerta de entrada pública. Los microservicios internos no están expuestos a Internet directamente, sino que operan de forma segura dentro de una red virtual privada (VPC).

Las imágenes almacenadas en S3 serán procesadas en diferentes tamaños (miniatura, mediana, grande) para optimizar su visualización según el contexto: las miniaturas para vistas previas, el tamaño mediano para el feed principal y el tamaño grande para la vista de detalle. Esto mejora el rendimiento y la experiencia de usuario.

Diagrama de seguridad

A continuación, se presenta el diagrama de secuencia que ilustra el flujo de autenticación y autorización del sistema Hipstagram. Este modelo detalla el proceso mediante el cual un usuario envía sus credenciales, la forma en que el API Gateway enruta la petición hacia el servicio de autenticación (Auth Service) para la validación y emisión del token JWT, y como las peticiones subsecuentes son autorizadas y delegadas a los microservicios correspondientes.



Usuario / Cliente · API Gateway (NGINX) · Microservicios (Auth · Post) · Mensaje / Petición · Respuesta · Lifeline (tiempo)

Hipstagram · Diagrama de Secuencia UML · Seguridad: JWT + bcryptjs + RBAC + API Gateway

Vista de implementación

La vista de implementación define como se empaquetan y despliegan los componentes de software de Hipstagram en la infraestructura de nube. Para garantizar la portabilidad y consistencia entre los entornos de desarrollo, pruebas y producción, el sistema utiliza una estrategia de despliegue basada en la contenedorización mediante Docker.

Toda la solución está proyectada para alojarse en la infraestructura de la nube (AWS), dividiendo la carga operativa en los siguientes recursos:

1. Cómputo (AWS EC2): Máquinas virtuales que ejecutaran los contenedores correspondientes al API Gateway y al ecosistema interno de microservicios.
2. Persistencia de datos relacionales (AWS RDS): Servicio gestionado de base de datos con el motor PostgreSQL, encargado de almacenar de forma segura los perfiles, publicaciones, interacciones y roles.
3. Almacenamiento de objetos estáticos (AWS S3): Servicio de almacenamiento escalable destinado exclusivamente para guardar los archivos multimedia (fotografías) que los usuarios suben a la plataforma.

Descripción General de la Implementación

La implementación del sistema Hipstagram se llevará a cabo siguiendo la arquitectura de microservicios definida en la fase de diseño. Cada microservicio será desarrollado como una aplicación independiente, empaquetada en un contenedor Docker, y se comunicará con los demás a través de API´s REST y mensajería asíncrona.

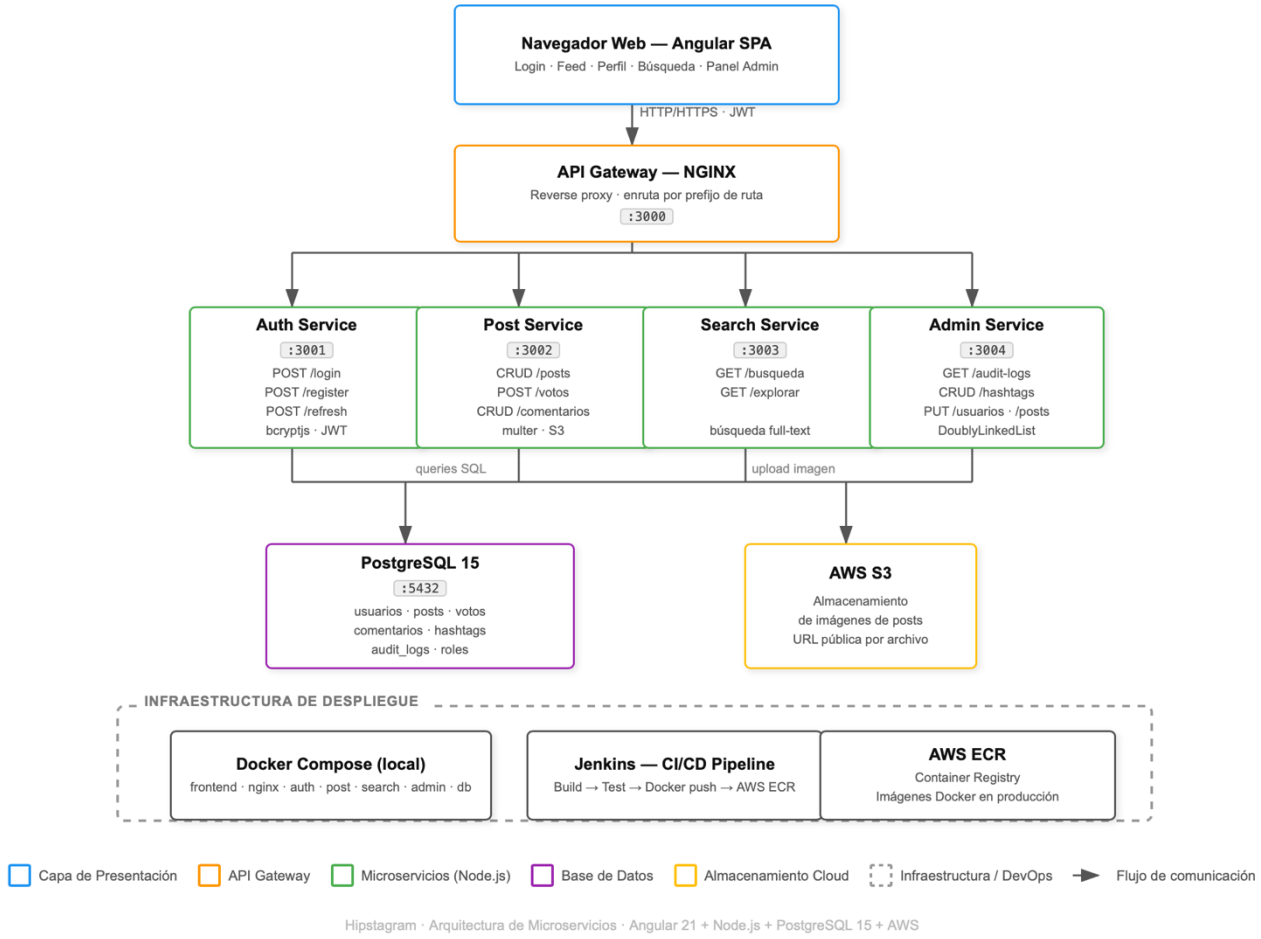
Estrategia de implementación:

- La siguiente tabla detalla las dependencias npm del backend, su función y la alternativa evaluada en caso de que la librería no esté disponible o sea incompatible con el entorno:

Paquete npm	**Capa / entorno**	**Función en el proyecto**	**Alternativa si falta**
express	Backend	Framework HTTP para definir rutas y middlewares.	Fastify (mejor rendimiento) o NestJS (más estructurado)
jsonwebtoken	Backend	Generación y verificación de tokens JWT (HS256).	jose (soporte RS256/ES256) o Passport.js

bcrypt	Backend	Hash seguro de contraseñas (cost factor 10).	argon2 (más resistente a ataques de hardware)
pg (node-postgres)	Backend	Cliente nativo PostgreSQL; pool de conexiones configurable.	Prisma ORM o TypeORM (agregan abstracción y migraciones)
multer + multer-s3	Backend	Carga de archivos multipart; almacena directamente en S3.	Cloudinary SDK o MinIO (almacenamiento local en desarrollo)
@aws-sdk/client-s3	Backend	SDK oficial AWS v3 para operaciones en buckets S3.	aws-sdk v2 (legacy) o Google Cloud Storage SDK
cors	Backend	Middleware CORS: controla los orígenes permitidos.	Cabeceras CORS manuales en el middleware de Express
typescript	Backend (build)	Tipado estático; compila a JS para el contenedor Docker.	JavaScript puro con JSDoc (pierde validación en tiempo de compilación)
jest + ts-jest	Backend (test)	Framework de pruebas unitarias con cobertura lcov.	Vitest (más rápido con ESM) o Mocha + Chai
eslint	Backend (CI)	Linter de calidad de código; ejecutado en el pipeline Jenkins.	Biome (linter + formatter en uno, más rápido)
@angular/core 17	Frontend	Framework SPA con componentes standalone y HttpClient.	React + Vite o Vue 3 (ecosistema más ligero)

Diagrama de la Implementación



El sistema Hipstagram está construido bajo una arquitectura de microservicios, donde cada componente se despliega de forma independiente en contenedores Docker. El cliente accede a la aplicación a través de un navegador web que ejecuta una SPA (Single Page Application) desarrollada en Angular. Todas las peticiones del cliente pasan por un API Gateway implementado con NGINX en el puerto 3000, el cual actúa como punto de entrada único y se encarga de redirigir el tráfico hacia el microservicio correspondiente según la ruta solicitada.

El sistema cuenta con cuatro microservicios desarrollados en Node.js con TypeScript: el Auth Service gestiona el registro e inicio de sesión de usuarios mediante tokens JWT; el Post Service administra la creación, eliminación, votos y comentarios de publicaciones, además de manejar la carga de imágenes hacia AWS S3; el Search Service permite buscar publicaciones por palabras clave y explorar hashtags populares; y el Admin Service proporciona funcionalidades exclusivas para administradores como la gestión de usuarios, moderación de posts y consulta del registro de auditoría.

Todos los microservicios comparten una base de datos relacional PostgreSQL 15, que almacena los datos de usuarios, publicaciones, votos, comentarios, hashtags y logs de auditoría. Las imágenes de los posts se almacenan en un bucket de AWS S3. Para el despliegue local se utiliza Docker Compose, mientras que el pipeline de integración continua con Jenkins automatiza la construcción y publicación de imágenes hacia AWS ECR.

Vista de puesta en marcha

Esta sección describe cómo instalar y poner en operación el sistema, tanto en entorno local (desarrollo) como en producción (AWS). Se detallan los prerrequisitos, las variables de entorno obligatorias y los accesos necesarios.

Prerrequisitos de software (instalar antes de comenzar):

1. Node.js 18 LTS o superior (runtime del backend).
2. Docker Desktop y Docker Compose (para ejecutar los microservicios en contenedores).
3. Git (para clonar el repositorio).
4. AWS CLI configurado con perfil que tenga permisos en S3, ECR, RDS y EC2 (solo producción).

Variables de entorno obligatorias (archivo .env en la raíz del proyecto):

Variable	Obligatoria	Descripción
DB_HOST	Sí	Host de PostgreSQL. En local: db. En producción: endpoint de AWS RDS.
DB_USER / DB_PASSWORD / DB_NAME	Sí	Credenciales de la base de datos PostgreSQL.
JWT_SECRET	Sí	Clave secreta para firmar y verificar tokens JWT. Mínimo 32 caracteres aleatorios.
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Sí	Credenciales IAM con permisos s3:PutObject, s3:GetObject y s3:DeleteObject en el bucket.
AWS_S3_BUCKET / AWS_REGION	Sí	Nombre del bucket S3 y región AWS (ej. us-east-1).
CORS_ORIGIN	No	Origen permitido para CORS. Default: http://localhost:4200. En producción: URL de AWS Amplify.

Pasos de instalación (entorno local):

5. Clonar el repositorio: git clone https://github.com/kazc93/Hipstagram
6. Crear el archivo .env en la raíz con las variables de la tabla anterior.
7. Levantar todos los servicios: docker-compose up --build
8. Verificar: GET http://localhost:3000/api/health debe retornar status ok.

Pasos de despliegue en producción (AWS EC2):

9. Hacer push al repositorio GitHub. Jenkins detecta el push automáticamente y ejecuta el pipeline (lint, test, SonarQube, build Docker, push a ECR).
10. El servidor EC2 debe tener el archivo /etc/hipstagram.env con las variables de entorno y acceso a Docker y AWS CLI.
11. El pipeline descarga la imagen del ECR y ejecuta: BACKEND_IMAGE=<ecr-url> docker-compose up -d --no-build
12. El init.sql se ejecuta automáticamente en el primer arranque del contenedor PostgreSQL (volumen postgres_data vacío). Crea todas las tablas, roles DB y permisos.

Accesos y permisos AWS requeridos:

13. IAM usuario/rol con permisos: ecr:GetAuthorizationToken, ecr:BatchGetImage, ecr:PutImage, s3:PutObject, s3:GetObject.
14. Repositorio privado en AWS ECR: hipstagram-backend (región us-east-1, cuenta 630171690893).
15. Grupo de seguridad EC2: puerto 3000 (backend) abierto hacia Internet. El frontend se despliega en AWS Amplify (acceso HTTPS gestionado por Amplify). La variable CORS_ORIGIN del backend debe incluir la URL de Amplify.

Vista de datos

La vista de datos define la estrategia de persistencia y organización de la información del sistema Hipstagram. El diseño se centra en garantizar la integridad, disponibilidad y seguridad de los activos digitales de los usuarios mediante un modelo de almacenamiento hibrido:

- Persistencia de Datos Estructurados: Se utiliza el sistema de gestión de bases de datos relacionales PostgreSQL (alojado en AWS RDS) para manejar toda la lógica de negocio que requiere consistencia de datos e integridad referencial.
- Persistencia de Archivos No Estructurados: Para optimizar el rendimiento de la base de datos, los archivos binarios pesados (fotografías) no se almacenan en tablas, sino en un servicio de almacenamiento de objetos AWS S3. En la base de datos únicamente se persiste la URL o el identificador del recurso.

- Consistencia e Integridad Referencial: Las FK desde posts hacia votos y comentarios tienen restricción RESTRICT. Si un usuario (rol USER) intenta eliminar un post con dependencias, el backend retorna HTTP 409 con el detalle de votos y comentarios existentes. Un administrador puede forzar la eliminación en cascade desde el endpoint DELETE /api/admin/posts/:id. Además, la inserción de hashtags al crear un post usa SAVEPOINT de PostgreSQL: si falla algún hashtag, solo se revierte esa parte sin cancelar la publicación completa.

Diagrama Entidad-Relación

El modelo de datos de Hipstagram ha sido diseñado bajo el principio de normalización para asegurar la integridad referencial y evitar la duplicidad de información. El siguiente diagrama representa la estructura de la base de datos relacional PostgreSQL, destacando las entidades de usuarios, roles de acceso y el motor de interacciones (publicaciones y likes). Se enfatiza la relación entre la tabla de publicaciones y el almacenamiento externo en AWS S3 mediante el campo de referencia de URL.

Diccionario de Datos

A continuación, se describen las especificaciones técnicas de las tablas que conforman la base de datos de Hipstagram:

Tabla: Roles

Almacena los niveles de acceso permitidos en el sistema.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_rol	SERIAL	No Nulo	PK	Identificador único del rol
nombre_rol	VARCHAR(30)	No Nulo		Nombre del nivel de acceso (USER o ADMIN)

Tabla: Usuarios

Contiene la información de cuenta y credenciales de los usuarios.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_usuario	SERIAL	No Nulo	PK	Identificador único del usuario.
username	VARCHAR(50)	No Nulo		Nombre de usuario único para la plataforma.
email	VARCHAR(100)	No Nulo		Correo electrónico de contacto.
password	TEXT	No Nulo		Hash de seguridad de la contraseña.
id_rol	INT	No Nulo	FK	Referencia al rol asignado (Tabla Roles).

Tabla: Publicaciones

Registra las fotos subidas por los usuarios y su metainformación.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_publicacion	SERIAL	No Nulo	PK	Identificador de la publicación.
id_usuario	INT	No Nulo	FK	Referencia al autor (Tabla Usuarios).
url_imagen	TEXT	No Nulo		Enlace al archivo en AWS S3.
descripcion	VARCHAR(128)	Nulo		Texto que acompaña la foto
fecha_creacion	TIMESTAMP	No Nulo		Fecha y hora del registro.
estado	VARCHAR(20)	No Nulo		Define si la foto es visible o está bloqueada.

Tabla: Reacciones

Gestiona las interacciones de votación entre los usuarios y las publicaciones, permitiendo el registro de reacciones tanto positivas (like) como negativas (dislike) de forma centralizada.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_voto	SERIAL	No Nulo	PK	Identificador de la interacción.
id_usuario	INT	No Nulo	FK	Referencia al usuario que realiza el voto
id_publicacion	INT	No Nulo	FK	Referencia a la publicación que recibe el voto.
tipo_voto	VARCHAR(10)	No Nulo		Define si es un Like o un Dislike

Restricción de Integridad: Se establece una restricción única compuesta (id_usuario, id_publicacion) para garantizar que un usuario solo pueda mantener una reacción activa por publicación, evitando la duplicidad de votos.

Tabla: Comentarios

Esta entidad registra las interacciones textuales que los usuarios realizan en las diversas publicaciones de la plataforma.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_comentario	SERIAL	No Nulo	PK	Identificador único del comentario
id_usuario	INT	No Nulo	FK	Referencia al autor del comentario
id_publicacion	INT	No Nulo	FK	Referencia a la publicación comentada
contenido	TEXT	No Nulo		Texto redactado por el usuario
fecha_creacion	TIMESTAMP	No Nulo		Fecha y hora en que se registró el comentario

Tabla: Hashtag

La tabla hashtag almacena las etiquetas utilizadas para clasificar y agrupar las publicaciones dentro del sistema. Permite organizar el contenido visual mediante palabras clave, facilitando la búsqueda, filtrado y categorización de publicaciones por parte de los usuarios.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_hashtag	INT	No Nulo	PK	Identificador único del hashtag.
nombre	VARCHAR(50)	No Nulo		Nombre del hashtag normalizado

Tabla: publicación_hashtag

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id_publicacion	INT	No Nulo	PK, FK	Referencia a publicación
Id_hashtag	SERIAL	No Nulo	PK, FK	Referencia a hashtag

Esta tabla actúa como entidad intermedia para gestionar la relación muchos a muchos entre Publicaciones y Hashtags. Permite asociar múltiples etiquetas a una misma publicación y, a su vez, vincular un hashtag con diversas publicaciones dentro del sistema.

Tabla: refresh_tokens

Almacena los tokens de refresco de sesión. Permite renovar el access token sin requerir nueva autenticación y soporta revocación individual de sesiones.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id	SERIAL	No Nulo	PK	Identificador único del token.
id_usuario	INT	No Nulo	FK	Referencia al usuario dueño del token.
token	TEXT	No Nulo		Valor del refresh token (UUID o JWT).
expires_at	TIMESTAMP	No Nulo		Fecha y hora de expiración del token (7 días).
created_at	TIMESTAMP	No Nulo		Fecha de creación del token.

Tabla: hashtags_prohibidos

Lista de palabras clave bloqueadas para la moderación automática de contenido. Cuando un post incluye un hashtag presente en esta tabla, su estado se establece en BLOQUEADO automáticamente.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id	SERIAL	No Nulo	PK	Identificador único.
nombre	VARCHAR(50)	No Nulo	UNIQUE	Nombre del hashtag bloqueado (normalizado, sin #).
agregado_por	INT	Nulo	FK	Referencia al administrador que agrego el hashtag (Tabla Usuarios).
fecha_agregado	TIMESTAMP	No Nulo		Fecha y hora en que se registro el hashtag prohibido. Por defecto: CURRENT_TIMESTAMP.

Tabla: audit_logs

Registra las operaciones sensibles del sistema para trazabilidad y auditoría. En memoria se mantiene una lista doblemente enlazada con los registros más recientes; la persistencia permanente se realiza en esta tabla PostgreSQL.

Campo	Tipo de Dato	Nulidad	Llave	Descripción
id	SERIAL	No Nulo	PK	Identificador único del registro.
request_id	UUID	No Nulo		Identificador de la petición HTTP (trazabilidad).
actor_user_id	INT	Nulo	FK	Usuario que ejecutó la acción (null para acciones del sistema).
actor_role	VARCHAR(20)	No Nulo		Rol del actor: USER, ADMIN o SYSTEM.
action	VARCHAR(50)	No Nulo		Tipo de acción (LOGIN, POST_CREATE,

			POST_DELETE, ADMIN_UPDATE, etc.).
entity_type	VARCHAR(30)	No Nulo	Entidad afectada (user, post, system).
result	VARCHAR(20)	No Nulo	Resultado de la operación: SUCCESS, FAILURE, BLOCKED.
ip_origen	VARCHAR(45)	Nulo	Dirección IP desde la que originó la solicitud.
created_at	TIMESTAMP	No Nulo	Marca de tiempo del evento.

Definición de roles de la aplicación

La plataforma Hipstagram utiliza un modelo de Control de Acceso Basado en Roles (RBAC) para gestionar los niveles de acceso dentro de los microservicios:

- Administrador (ADMIN): Posee facultades administrativas sobre el sistema. Puede gestionar usuarios (bloquear/desbloquear), moderar publicaciones, administrar la lista de hashtags prohibidos, consultar el log de auditoría y eliminar publicaciones en cascade (incluso si tienen votos o comentarios). Accede al panel administrativo en /api/admin.
- Usuario (USER): Es el rol estándar asignado a los clientes. Tiene capacidades para gestionar su propio perfil, crear publicaciones con imagen en AWS S3, redactar descripciones con hashtags, realizar votos like/dislike y agregar comentarios. No puede eliminar publicaciones que tengan votos o comentarios (el backend retorna HTTP 409).

Limitaciones del proyecto:

- Tipo de archivos: El sistema está restringido únicamente a imágenes en formatos estándar (JPG, PNG), sin soporte para video o audio en esta versión.
- Dependencia de Red: La aplicación requiere una conexión a internet constante para interactuar con los servicios gestionados de AWS; no cuenta con modo offline.
- Notificaciones: No se incluye el desarrollo de un motor de notificaciones push o alertas en tiempo real para estas fases del proyecto.